



University of Piraeus

School of Information and Communication Technologies

Department of Digital Systems

Level: MSc Programme

Advanced Topics in Cybersecurity and Artificial Intelligence

Hybrid Intrusion Detection with Snort and Machine Learning

Supervisors: Christos Xenakis, Apostolis Zarras and Georgios Vouros

Ioannis Kalaitzidis

ioannis_kalaitzidis@ssl-unipi.gr

Piraeus

05/07/2026

Introduction

Every organization's network faces daily intrusion attempts, from automated scans to targeted attacks. Because the volume of traffic is enormous and attacks unfold at machine speed, manual inspection is practically impossible. This job is handled by an Intrusion Detection System, an automated mechanism that monitors a network's traffic and raises an alert when it spots something hostile. This monitoring is essentially a data problem, since a tiny amount of malicious activity must be told apart within a huge volume of normal traffic, dealing with the classic challenges of volume, speed and noise (lecture 1).

The aim of this work is not merely to demonstrate that such a system detects attacks, but to systematically measure how well it does so, by what means and within what limits. To this end, a controlled lab environment was set up, realistic legitimate and malicious traffic was produced, two different detection technologies were applied, and they were then combined into a single hybrid system.

There are two large families of detection techniques, and the whole work rests on the difference between them. The first is called signature-based detection. It works like a classic antivirus, which recognizes a virus because it knows its fingerprint in advance. It relies on explicit rules of the form: if a specific pattern is found, an alert is raised. It is very precise, since it does not raise an alert for traffic that does not match a known pattern, but it has a fundamental weakness that the theory also stresses (lecture 4): it does not detect anything it has not seen before, nor attacks that hide on purpose.

The second family is called anomaly-based or machine-learning detection. Instead of explicit rules, the system learns from examples what normal traffic looks like and flags whatever deviates from it. Its advantage is that it can also catch unknown attacks. Its drawback is that it also fires on unusual but legitimate behavior, producing many false alarms and the well-known alert fatigue, that is, the tiredness of analysts from the many false alerts (lecture 1).

The contrast between the two approaches is captured by an analogy of two guards at an entrance. The first recognizes only specific, recorded individuals as intruders. He stops with certainty anyone on the list, but lets an unknown intruder in disguise pass. The second judges on the basis of unusual behavior. He spots the unknown one, but also bothers innocent people who simply act strangely. Neither is sufficient on its own. The solution is their cooperation, with clear rules for the cases where they disagree. This is exactly the hybrid system built in this work.

Particular emphasis was placed on the honesty of the evaluation, because of a pitfall that, if not taken into account, leads to impressive but fake results. If a model is scored using as the correct answers the same data given to it to decide, then it simply repeats what it already knows and produces

a perfect score, which however has no value. The situation corresponds to handing an examinee the exam questions together with their solutions. To avoid it, the labeling of each time interval as an attack (the correct answer) was designed to come from a source entirely independent of the data the model processes. Thanks to this, the reported metrics are realistic (for example, the hybrid system's F1 score is 0.926) and reflect real detection ability. This stance is connected to the principles of responsible artificial intelligence (lecture 7). We observed three:

- **Interpretability:** the system explains why it made each decision. It shows which traffic features weighed most and gives a risk score at every moment, instead of acting as an opaque black box.
- **Accountability:** every alert is accompanied by its source, that is, which side (the rule or the model) triggered it, so that it can be audited.
- **Honest reporting:** we did not hide the system's failures (such as the model's false positives) nor did we embellish the results.

As the application domain we chose a web server inside an internal network, which receives attacks from an external attacker. This scenario is realistic and combines a clear legitimate function (serving web pages) with a broad range of network attacks.

The work answers seven tasks. We will design the lab and describe the adversary (Task 1), produce and label the traffic (Task 2), build Snort rules (Task 3), train machine-learning models (Task 4), combine the two layers (Task 5), evaluate the system (Task 6) and study how an attacker would try to fool it (Task 7). We also implemented two optional tasks. The following sections explain each of the above in detail.

Task 1: Testbed design and threat model

Before one can detect attacks, one needs a safe place to run and study them. This is called a testbed, that is, a controlled lab environment in which we know exactly what happens at every moment. Because we ourselves set up both the attacks and the legitimate traffic, we know with certainty which moment was an attack and which was not, something that is essential for a fair evaluation.

Topology and target service

We set up two separate networks instead of one. The internal network we call HOME_NET and it corresponds to the address range 10.0.0.0/24 (that is, the addresses that start with 10.0.0). Inside it sits the victim server at address 10.0.0.10, which serves web pages on port 8080. A port is like a door number. A computer can run many services at once, and the port distinguishes which service each request is addressed to. On the same internal network there is also a legitimate user (10.0.0.50). The external network, which we call EXTERNAL_NET (that is, anything not belonging to HOME_NET), includes the attacker (192.168.56.20) and a legitimate external visitor (203.0.113.7).

The reason we chose two networks and not one is substantial. If everything ran on the same machine (on localhost), then the notion of traffic coming from outside would make no sense, since everything would originate from the same address. One of our rules detects a ping coming from outside the internal network. With two networks, the attacker is genuinely outside, so the rule has real meaning and can be tested.

Threat model: who we are facing

To harden a system, we must first describe the adversary. Our attacker can send any traffic he wants to the server from the external network, that is, packets of every kind and web requests. He does not, however, have privileged access; he does not control the server, nor the Snort sensor, nor the models or the data they were trained on. His goal is to reconnoiter the system (to see what is running and where), to overload it, and to attack the web application. We define a breach as the presence of malicious activity within a time interval, while correct operation is defined as detecting the attack without blocking legitimate traffic.

How we know what was an attack (ground truth)

To say whether the system answered correctly, we need the objective truth, that is, which moments were actually an attack. This truth we call ground truth and we have it from two independent sources. First, we scheduled when each attack runs and recorded the exact times in a file (logs/event_log.csv). Second, we know the attacker's address. We split time into one-second pieces, which we call windows, and we label a window as an attack if it contains at least one packet from the attacker, otherwise as legitimate.

Why we kept the truth separate from what the model sees

Here lies the most important point of our methodology. The truth (whether a window was an attack) is determined solely by the time and the attacker's address. It is never determined by the measurable features of the traffic, such as the number of packets or their flags.

The reason is the following. If the labeling of a window as an attack came from a rule of the form, if the packets exceed the number X then it is an attack, and the model were then asked to tell attacks apart by looking at exactly that packet count, then it would simply re-learn the same rule. It would produce a perfect score, but a completely fake one, since it would not have learned anything real. This problem is called data leakage. By keeping the labeling independent of the features, the metrics are real, genuine errors appear (both missed attacks and false alarms), and the whole analysis that follows becomes meaningful.

How the system is built

The system is not a single large program, but a series of small programs in Python, each with one job, like the steps of a production line. `build_dataset.py` produces the traffic, `feature_extraction.py` computes the features and labels, `snort_emulator.py` implements the signature side, `train_model.py` trains the models, `hybrid_fusion.py` combines the two signals and `evaluation.py` produces the metrics and charts. The whole chain runs with a single command through `run_all.sh`.

Why the results come out the same every time

The traffic generation and the model training use a fixed randomness number (the number 1337). This number makes the randomness repeatable, so that every run gives exactly the same results. In addition, all analysis is done offline over a single traffic capture file (pcap), that is, over a recording of the traffic that we analyze again and again, instead of relying on a live capture that would change each time.

Log files (logs/)

The `logs/` folder holds the system's evidence. Table 1 explains what each file contains and what it is used for.

File	Columns	Role
event_log.csv	attack_type, start_time, end_time, description	When each attack ran. On this the labeling of the windows rests. feature_extraction.py reads it.
snort_alerts.csv	rel_time, sid, msg, src	Snort's alerts, one per packet. It proves that each rule fires.
snort_windows.csv	window_id, snort_detection, sids_fired	Snort's signal per window. It is the bridge to the model. hybrid_fusion.py joins them via window_id.

Files implementing Task 1: the documentation file TESTBED_DESIGN.md, which describes the lab topology and the methodology, and the program traffic_generation/build_dataset.py, which when executed produces two files: the traffic file pcaps/lab_traffic.pcap (the recording of all the traffic) and the attack timeline logs/event_log.csv (the ground truth).

Task 2: Traffic generation and labeling

A detection system cannot be evaluated without data. We therefore produced a realistic traffic stream lasting about 1,799 seconds (around half an hour), containing 24,110 packets. A packet is the smallest unit of data that travels on the network, like an envelope with a little information inside. Of these, 22,456 are TCP packets (the protocol on which the web is based) and 1,654 are ICMP (the protocol of pings).

The legitimate traffic

The legitimate traffic simulates the everyday life of a server. It includes normal visits to web pages from the internal and the external legitimate user, periodic pings to check that the server is alive, and every so often some heavy but perfectly legitimate bursts, such as when a page loads many images at once. We deliberately added these heavy bursts so that the problem is not artificially easy. If all legitimate traffic were quiet and all attacks noisy, the separation would be trivial and the results misleading.

The attacks

We implemented six types of attacks, covering the required four malicious scenarios plus one low-and-slow attack, and adding a stealth variant to clearly show the complementarity of the detectors. For each type we explain what it is and why an attacker does it:

- **SYN scan:** the attacker sends connection requests to a great many ports, as if knocking on all the doors one by one to see which are open. It is the first step of reconnaissance.
- **ICMP flood:** floods the server with high-frequency pings, trying to overload it.
- **Connection flood:** opens a great many full connections very quickly, exhausting the server's resources.
- **HTTP probe:** requests suspicious addresses (e.g. admin pages such as /phpmyadmin/) and uses the identifier of a known attack tool (sqlmap), leaving a characteristic fingerprint.
- **Stealth web probe:** makes the same suspicious requests, but sparsely and individually, so that the volume looks normal and passes unnoticed.
- **Low-and-slow SQLi:** sends an SQL injection attack cut into small pieces with delays between them, so that it stays below the thresholds that trigger the alerts. It is specifically designed to evade.

From packets to windows and class imbalance

We do not examine each packet separately, but group the packets into one-second windows and describe each one with aggregate quantities (e.g. how many packets, how many pings, etc.). This yields 1,484 windows, of which 174 are attack and 1,310 legitimate. That only 11.7% are attacks is intentional and realistic, because in reality attacks are rare. This imbalance matters, because it forces us not to look only at overall accuracy. If someone always said there is no attack, they would be right in 88% of cases, without however having caught a single attack. That is why we use more appropriate metrics, as we explain in Task 6 (lecture 5).

Attack type	Episodes	Attack windows
SYN_SCAN	4	8
ICMP_FLOOD	3	12
CONN_FLOOD	3	8

Attack type	Episodes	Attack windows
HTTP_ATTACK	4	19
STEALTH_WEB	2	38
SLOW_ATTACK	2	89
Total	18	174

Files implementing Task 2: the program `traffic_generation/build_dataset.py` produces the traffic, and `ml_detector/feature_extraction.py` processes this traffic and produces the file `features.csv`, that is, the table with the features of each window together with its label (attack or legitimate).

Task 3: Development of Snort rules

Snort is a widely used, open-source intrusion detection system that works with signatures. It reads the network's traffic and compares it against a list of rules. Each rule essentially states that, if a specific form of traffic is found, the corresponding alert is recorded. In this work five rules were implemented (in the file `rules/local.rules`), more than the three that were required.

A rule has two basic parts. The first is what it searches for within the content, for example a specific word in a web request (this is called a content match). The second is a frequency threshold (detection_filter), that is, the triggering of an alert only when something happens many times in a short time. The threshold is important, because a single ping is normal, whereas hundreds of pings per second constitute an attack. The rules that were developed are:

- **sid 1000001:** catches a request to a suspicious admin page (`/phpmyadmin/`) based on the content, without a threshold, so that even a single such request is detected.
- **sid 1000002:** catches the fingerprint of the `sqlmap` tool in the request header.
- **sid 1000003:** catches a barrage of pings coming from outside, with a threshold of 30 per second per source.
- **sid 1000004:** catches port scanning, with a threshold of 40 connection requests in 2 seconds per source.
- **sid 1000005:** catches the connection flood, with a threshold of 20 per second per source.

To confirm that they work, we ran the rules over the recorded traffic and counted the alerts. All five fire, as the table below shows. We must clarify a practical detail. So that anyone can reproduce the results without installing Snort, we embedded in the pipeline a faithful emulator of the rules (`snort_emulator.py`) that applies exactly the same thresholds. The real Snort runs once, separately,

only to capture the alert screenshots (instructions in SNORT_GUIDE.md), and its results agree with the emulator's.

sid	Rule	Alerts
1000001	WEB Suspicious URI (phpmyadmin)	46
1000002	WEB sqlmap User-Agent	95
1000003	ICMP Echo flood (external)	1415
1000004	TCP SYN scan / port sweep	111
1000005	TCP connection flood (burst)	171

Verification with real Snort

Beyond the emulator, the rules were also verified with real Snort 3.12.2 in a Kali Linux environment, running it offline over the same pcap. All 24,110 packets were analyzed and a total of 1607 alerts were fired. The three volume rules fired normally (ICMP flood 1412, SYN scan 73, connection flood 122), while the two content rules (phpmyadmin, sqlmap) did not fire in this version, since the http_inspect mechanism of Snort 3.12 does not match the synthetic HTTP payload the way the emulator does. The relevant screenshots are provided in the screenshots folder. The metrics of the machine learning and of the hybrid system are not affected, since they derive from the full set of rules through the emulator.

Running real Snort 3.12.2 on Kali Linux over the pcap: loading of the five rules and full analysis summary (24,110 packets, 1607 alerts):

```
(kali@kali:~/Downloads/XYZ2022_Snort)
$ snort -v snort_kali.lua -i rules_kali.rules -r pcap/lab_traffic.pcap -A alert_fast -l logs -b none

v")= Snort++ 3.12.2.0

Loading snort_kali.lua:
  http_inspect
  alert_fast
  output
  ips
  stream_tcp
  search_engine
  packets
  active
  alerts
  dq
  decode
  host_cache
  host_tracker
  hosts
  process
  network
  stream
  stream_icmp
  binder
Finished snort_kali.lua:
Loading rule args:
Loading rules_kali.rules:
Finished rules_kali.rules:
Finished rule args:

-----
ips policies rule stats
-----
  id loaded shared enabled % file
  -- -- -- -- --
  1 1 0 1 100 snort_kali.lua

rule counts
-----
  total rules loaded: 5
  text rules: 5
  option chains: 5
  chain headers: 2

port rule counts
-----
  tcp udp icmp ip
  any 0 0 1 0
  dst 2 0 0 0
  total 2 0 1 0

service rule counts
-----
  http: 2 0
  https: 2 0
  http3: 2 0
  total: 6 0

fast pattern groups
-----
  to_server: 0

search engine (as_bfpa)
-----
  instances: 0
  patterns: 0
  pattern chars: 54
  num states: 54
  num match states: 0
  memory scale: KB
  total memory: 0.22656
  pattern memory: 0.28125
  match list memory: 0.00000
  transition memory: 0.51000
  fast pattern only: 0

pcap dq configured to read-file.
Commencing packet processing
Entry queue interval is: 100 us
+ [4] pcap/lab_traffic.pcap
- [4] pcap/lab_traffic.pcap

-----
Packet Statistics
-----
dq
  Decs: 1
  processed: 24110
  analyzed: 24110
  allow: 24110
  rx_bytes: 431466

-----
Codec
-----
  total: 24110 (100.00%)
  icmp: 1074 ( 4.45%)
  ipva: 24110 (100.00%)
  tcp: 23036 ( 95.55%)

-----
Module Statistics
-----
binder
  raw_packets: 4167
  raw_flow: 2356
  no_match: 1007
  inputs: 0000

-----
Detection
-----
  analyzed: 24110
  hard_evals: 10000
  alerts: 1607
  total_alerts: 1607
  logged: 1607

-----
ips_actions
-----
  alert: 1607

-----
normalizer
-----
  test_tcp_syn_min: 0000
  test_tcp_syn_data: 00

-----
search_engine
-----
  non_qualified_events: 9464
  qualified_events: 1466

-----
stream
-----
  flows: 2356
  total_sessions: 21
  idle_sessions_timeout: 3
  flows_closed: 16
  icmp_timeout_sessions: 0
  no_flow_tcp_reset: 011
  no_flow_unwanted: 2326

-----
stream_icmp
-----
  sessions: 4
  max: 4
  created: 4
  released: 4

-----
stream_tcp
-----
  sessions: 2354
  max: 2354
  created: 2354
  released: 2354
  timeout: 16
  instantiated: 2354
  setups: 2354
  syn_ack_tracker: 2354
  seqs_queued: 2400
  seqs_released: 2400
  overlaps: 113
  paylnd_fully_trimmed: 48
  syn_ack: 2354
  flags: 1306
  max_seqs: 3
  max_bytes: 200
  tcp_min_probes: 4167

-----
Summary Statistics
-----
timing
  runtime: 00:00:00
  seconds: 0.180730
  pkts/sec: 133467
  alerts/sec: 107

v")= Snort exiting
```

Sample of the alerts (logs/alert_fast.txt) and number of alerts per rule:

```
(kali@kali)~/Downloads/MTE25012_Snort
$ cat logs/alert_fast.txt | head -30
for s in 1000001 1000002 1000003 1000004 1000005; do echo -n "$s: "; grep -c $s logs/alert_fast.txt; done
05/23-17:38:23.469839 ** [1:1000003:1] "ICMP Echo flood from external host" ** [Priority: 0] [ICMP] 192.168.56.20 → 10.0.0.10
05/23-17:38:23.475039 ** [1:1000003:1] "ICMP Echo flood from external host" ** [Priority: 0] [ICMP] 192.168.56.20 → 10.0.0.10
05/23-17:38:23.481839 ** [1:1000003:1] "ICMP Echo flood from external host" ** [Priority: 0] [ICMP] 192.168.56.20 → 10.0.0.10
05/23-17:38:23.487639 ** [1:1000003:1] "ICMP Echo flood from external host" ** [Priority: 0] [ICMP] 192.168.56.20 → 10.0.0.10
05/23-17:38:23.493839 ** [1:1000003:1] "ICMP Echo flood from external host" ** [Priority: 0] [ICMP] 192.168.56.20 → 10.0.0.10
05/23-17:38:23.499839 ** [1:1000003:1] "ICMP Echo flood from external host" ** [Priority: 0] [ICMP] 192.168.56.20 → 10.0.0.10
05/23-17:38:23.505839 ** [1:1000003:1] "ICMP Echo flood from external host" ** [Priority: 0] [ICMP] 192.168.56.20 → 10.0.0.10
05/23-17:38:23.511839 ** [1:1000003:1] "ICMP Echo flood from external host" ** [Priority: 0] [ICMP] 192.168.56.20 → 10.0.0.10
05/23-17:38:23.517839 ** [1:1000003:1] "ICMP Echo flood from external host" ** [Priority: 0] [ICMP] 192.168.56.20 → 10.0.0.10
05/23-17:38:23.523839 ** [1:1000003:1] "ICMP Echo flood from external host" ** [Priority: 0] [ICMP] 192.168.56.20 → 10.0.0.10
05/23-17:38:23.529839 ** [1:1000003:1] "ICMP Echo flood from external host" ** [Priority: 0] [ICMP] 192.168.56.20 → 10.0.0.10
05/23-17:38:23.535839 ** [1:1000003:1] "ICMP Echo flood from external host" ** [Priority: 0] [ICMP] 192.168.56.20 → 10.0.0.10
05/23-17:38:23.541839 ** [1:1000003:1] "ICMP Echo flood from external host" ** [Priority: 0] [ICMP] 192.168.56.20 → 10.0.0.10
05/23-17:38:23.547839 ** [1:1000003:1] "ICMP Echo flood from external host" ** [Priority: 0] [ICMP] 192.168.56.20 → 10.0.0.10
05/23-17:38:23.553839 ** [1:1000003:1] "ICMP Echo flood from external host" ** [Priority: 0] [ICMP] 192.168.56.20 → 10.0.0.10
05/23-17:38:23.559839 ** [1:1000003:1] "ICMP Echo flood from external host" ** [Priority: 0] [ICMP] 192.168.56.20 → 10.0.0.10
05/23-17:38:23.565839 ** [1:1000003:1] "ICMP Echo flood from external host" ** [Priority: 0] [ICMP] 192.168.56.20 → 10.0.0.10
05/23-17:38:23.571839 ** [1:1000003:1] "ICMP Echo flood from external host" ** [Priority: 0] [ICMP] 192.168.56.20 → 10.0.0.10
05/23-17:38:23.577839 ** [1:1000003:1] "ICMP Echo flood from external host" ** [Priority: 0] [ICMP] 192.168.56.20 → 10.0.0.10
05/23-17:38:23.583839 ** [1:1000003:1] "ICMP Echo flood from external host" ** [Priority: 0] [ICMP] 192.168.56.20 → 10.0.0.10
05/23-17:38:23.589839 ** [1:1000003:1] "ICMP Echo flood from external host" ** [Priority: 0] [ICMP] 192.168.56.20 → 10.0.0.10
05/23-17:38:23.595839 ** [1:1000003:1] "ICMP Echo flood from external host" ** [Priority: 0] [ICMP] 192.168.56.20 → 10.0.0.10
05/23-17:38:23.601839 ** [1:1000003:1] "ICMP Echo flood from external host" ** [Priority: 0] [ICMP] 192.168.56.20 → 10.0.0.10
05/23-17:38:23.607839 ** [1:1000003:1] "ICMP Echo flood from external host" ** [Priority: 0] [ICMP] 192.168.56.20 → 10.0.0.10
05/23-17:38:23.613839 ** [1:1000003:1] "ICMP Echo flood from external host" ** [Priority: 0] [ICMP] 192.168.56.20 → 10.0.0.10
05/23-17:38:23.619839 ** [1:1000003:1] "ICMP Echo flood from external host" ** [Priority: 0] [ICMP] 192.168.56.20 → 10.0.0.10
05/23-17:38:23.625839 ** [1:1000003:1] "ICMP Echo flood from external host" ** [Priority: 0] [ICMP] 192.168.56.20 → 10.0.0.10
05/23-17:38:23.631839 ** [1:1000003:1] "ICMP Echo flood from external host" ** [Priority: 0] [ICMP] 192.168.56.20 → 10.0.0.10
05/23-17:38:23.637839 ** [1:1000003:1] "ICMP Echo flood from external host" ** [Priority: 0] [ICMP] 192.168.56.20 → 10.0.0.10
05/23-17:38:23.643839 ** [1:1000003:1] "ICMP Echo flood from external host" ** [Priority: 0] [ICMP] 192.168.56.20 → 10.0.0.10
1000001: 0
1000002: 0
1000003: 1412
1000004: 73
1000005: 122
(kali@kali)~/Downloads/MTE25012_Snort
```

Number of alerts per rule (sid): ICMP flood 1412, SYN scan 73, connection flood 122:

```
(kali@kali)~/Downloads/MTE25012_Snort
$ for s in 1000001 1000002 1000003 1000004 1000005; do echo -n "$s: "; grep -c $s logs/alert_fast.txt; done
1000001: 0
1000002: 0
1000003: 1412
1000004: 73
1000005: 122
(kali@kali)~/Downloads/MTE25012_Snort
```

Files implementing Task 3: the file rules/local.rules contains the five Snort rules and rules/snort.lua the configuration for running them, while the program ml_detector/snort_emulator.py applies the rules to the traffic and produces two files: logs/snort_alerts.csv with the per-packet alerts and logs/snort_windows.csv with the per-window Snort signal.

Task 4: Features and machine-learning models

What features are

A machine-learning model does not see raw packets, it sees numbers. A feature is a number that describes something about a one-second window. We computed 17 such features, chosen so as to relate to the attacks. Roughly, they fall into groups: volume quantities (how many packets and how many bytes, average packet size), counts of the TCP flags (small indications that show whether a packet opens, closes or transfers data over a connection), ping count, how many different senders and how many different ports appeared, and timing features (how far apart in time the packets are from

one another). The timing features are especially useful for slow attacks, which stand out not by their volume but by their rate.

How a model is trained and how it is tested

We split the data into two parts, 70% for training and 30% for testing. The model learns only from the first part and is tested on the second, which it has not seen before, exactly as a student studies some topics and is examined on new ones. We made sure the split preserves the same attack ratio in both parts. Some models need the features on a common scale, which is why we normalize them, and we learned this normalization exclusively from the training part, so that no information leaks from the test part (lecture 4). All the metrics we report come from the test part (446 windows, of which 52 attacks).

The three models

We tried three models, all from the course material (lectures 3 and 4). Random Forest is an ensemble of many small decision trees that vote and is considered the most reliable starting point for tabular data (lecture 4). Logistic Regression is a simple weighted formula that is easy to interpret. Isolation Forest is of a different philosophy; it needs no attack examples at all, it only learns what normal looks like and flags whatever stands out as an anomaly.

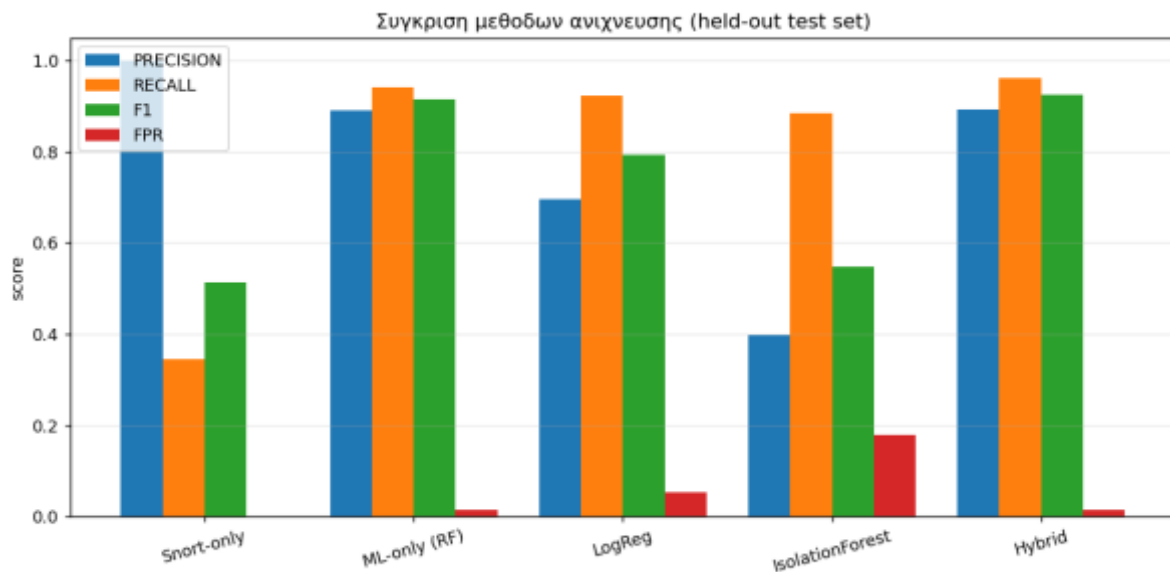
Before we read the results, four concepts are needed. Precision answers, of those the system flagged as an attack, how many were actually an attack. Recall answers, of the attacks that actually happened, how many the system caught. F1 is a balance of the two in a single number. The false positive rate (FPR) shows how often it rings an alarm on legitimate traffic. Ideally we seek high precision and high recall at the same time.

Model	Precision	Recall	F1	FPR	AUC
Random Forest	0.891	0.942	0.916	0.015	0.991
Logistic Regression	0.696	0.923	0.793	0.053	0.968
Isolation Forest	0.397	0.885	0.548	0.178	0.908

Random Forest gives the best balance (F1=0.916). Logistic Regression catches almost all attacks, but also fires wrongly more often (FPR=0.053), something expected for a simple linear model when

the data are not separable by a straight line. Isolation Forest, because it never saw attack examples, has good recall but low precision (0.397), that is, it flags many normal but unusual windows as anomalies. This confirms a basic observation, that the unusual is not necessarily malicious (lecture 3). It is worth stressing that no metric is perfect, a fact that is a positive sign, since it confirms the avoidance of data leakage and the measurement of real ability.

We also examined which features weighed most in Random Forest's decisions. At the top were the timing features (how far apart the packets are from one another), the number of different senders and the number of packets, which fits perfectly with the nature of the attacks. The figure below visually compares all the methods:



Files implementing Task 4: the program `ml_detector/feature_extraction.py` computes the features of each window, and `ml_detector/train_model.py` trains the models. Its execution produces `predictions.csv` with the predictions, `ml_metrics.json` with the metrics, as well as the trained models in files (`rf_model.pkl` the Random Forest, `scaler.pkl` the normalizer and `iforest.pkl` the Isolation Forest).

Task 5: Hybrid Fusion

Fusion means combining the opinions of the individual detectors into a single final decision. We gave each window a risk score, combining three signals with weights: $\text{risk} = 0.55 \times (\text{Random Forest score}) + 0.30 \times (\text{Snort signal}) + 0.15 \times (\text{Isolation Forest score})$

The weights are not arbitrary. We gave the largest weight to Random Forest because it was the most reliable model, medium weight to Snort because it is very precise when it fires, and the smallest weight to Isolation Forest because it is the noisiest. The risk score we translate into three levels, like a traffic light: low (LOW), medium (MEDIUM) and high (HIGH).

The final decision, that is, whether an alert will be raised, is defined as the union of the two most precise detectors, that is, we fire if Snort fires or if Random Forest says attack. This way each covers the other's blind spot. Isolation Forest does not fire on its own, because we would raise many false alarms. It only serves to raise the severity level, as a kind of watch.

What happens when the two detectors disagree

We defined explicit rules for the disagreements, because that is where the value of a hybrid system is decided:

- **C1, when the signature speaks:** if Snort fires but the model has a low score, we trust the rule (the signature is a sure indication) and raise the level to at least HIGH.
- **C2, when only the anomaly speaks:** if neither Snort nor Random Forest fires, but Isolation Forest shows a strong anomaly, we put the window at MEDIUM level for monitoring, with lower certainty.
- **C3, when only the model speaks:** if Random Forest fires while Snort is silent, we trust the model. This is exactly where the slow attack is caught, in which Snort is blind because the content arrives fragmented.

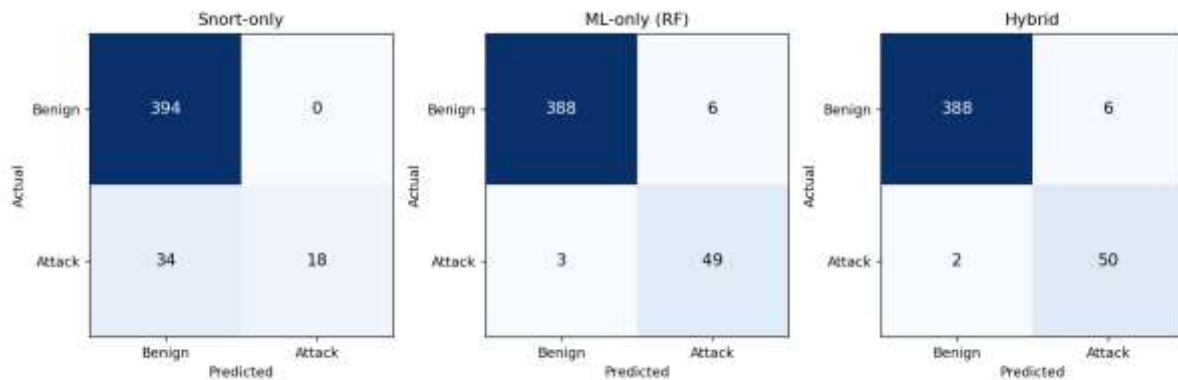
Files implementing Task 5: the program `ml_detector/hybrid_fusion.py` combines the Snort signal with the models' scores and produces `hybrid_results.csv`, which contains the risk score and the final decision for each window, as well as `conflicts.json`, which records the cases of disagreement between the two detectors.

Task 6: Evaluation

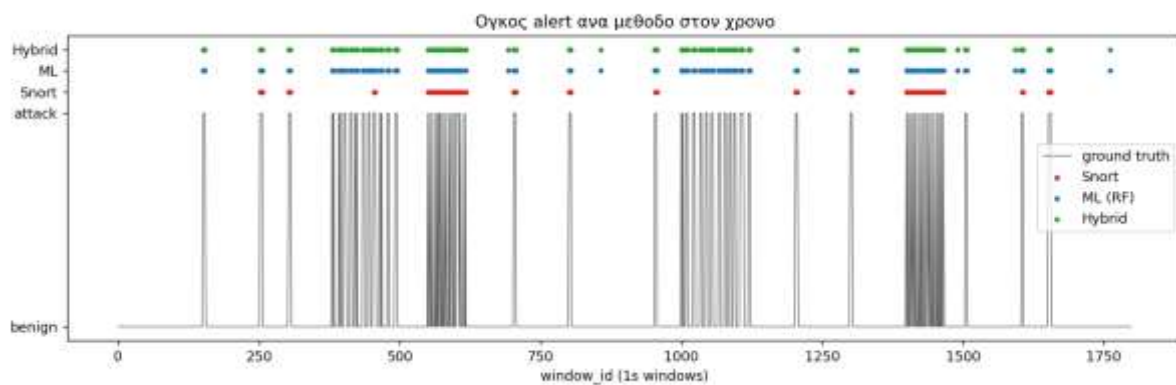
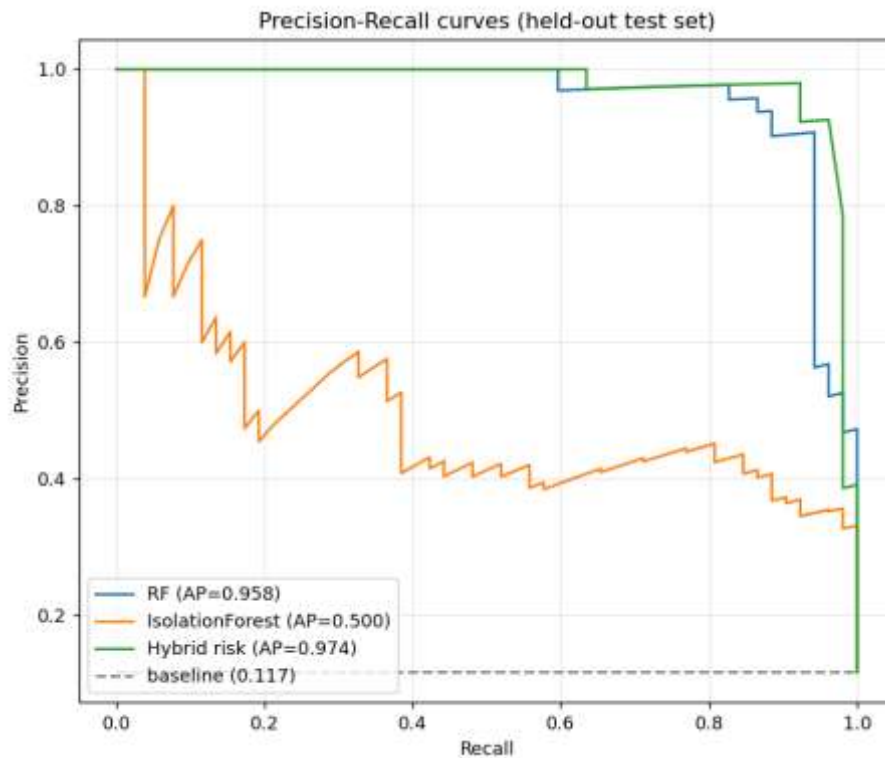
Below is the comparison of the three approaches in terms of their performance. Table 5 compares the three on the test set. Snort on its own has perfect precision (1.000), that is, it never fires wrongly, but low recall (0.346), that is, it misses most attacks, especially the slow and the stealthy ones. The model on its own (Random Forest) sharply raises the recall. The hybrid achieves the best F1 (0.926), catching one additional attack compared with Random Forest, without losing precision.

Method	TP	FP	FN	Precision	Recall	F1	FPR
Snort-only	18	0	34	1.000	0.346	0.514	0.000
ML-only (RF)	49	6	3	0.891	0.942	0.916	0.015
Hybrid	50	6	2	0.893	0.962	0.926	0.015

The columns TP, FP, FN are worth explaining, because they are the core of every evaluation (lecture 5). TP (true positive) is an attack that was caught correctly. FN (false negative) is an attack that escaped. FP (false positive) is a false alarm on legitimate traffic. The ideal system has many TP and very few FN and FP. The figure below shows the so-called confusion matrices, which are simply this count in table form for each method.



Because attacks are rare, we emphasized the precision-recall curves instead of the more common ROC curves, since the former are more informative when one class is rare (lecture 5). The first figure shows that the hybrid risk score is superior (average precision AP=0.974). The second figure shows when each method fires over time. It turns out that Snort fires rarely but correctly, while the hybrid also covers the long periods of the slow attack that Snort ignores.



Three cases where the detectors disagree

The following cases come from real windows and clearly show why no detector suffices on its own.

- **The model catches, Snort misses:** in 98 windows Random Forest catches attacks that Snort misses. These are mainly the slow attack (fragmented content and a rate below the thresholds) but also the beginning of the port scan. Characteristic is window 151, with 122 different ports. The model catches it immediately, while Snort has not yet had time to accumulate enough requests to exceed its threshold.

- **Snort catches, the model misses:** Snort catches stealthy requests that the model misses. In window 1428 the model's score is only 0.12, because the volume looks like heavy but legitimate browsing. Snort, however, recognizes the sqlmap fingerprint within the content and fires. Thanks to rule C1, the hybrid classifies it as HIGH.
- **False positives of the model:** in 7 windows Random Forest fires wrongly while Snort stays clean (e.g. in windows 692 and 857, with heavy but legitimate traffic). Snort's perfect precision balances the model's tendency toward false positives.

How much to trust the numbers

The metrics must be read with awareness of the sample size. With 52 attacks in the test set, each misclassification moves the recall by about two points. Therefore, the small differences between Random Forest and the hybrid should not be over-emphasized. The essential difference is seen more clearly in the analysis of the disagreements over the whole time series, where the model catches 98 windows on its own and Snort one. That no metric reaches the perfect 1.000 is a sign of sound methodology, not of weakness (lecture 4).

Files implementing Task 6: the program `ml_detector/evaluation.py` computes the metrics and `evaluation.json` with the detailed results, while the file `CASE_STUDIES.md` documents the three case studies.

Task 7: Evasion techniques and critical assessment

A clever attacker does not attack blindly, he tries not to be noticed. We examined three evasion techniques and which detector each one fools:

- **Content fragmentation (slow attack):** the signature breaks into many packets, so that none contains it whole. It mainly fools Snort, which searches the content, not the model, which looks at statistics.
- **Hiding within low volume:** sparse, individual requests so that the volume looks normal. It mainly fools the model, which relies on aggregate quantities, while Snort still catches the content.

- **Staying below the thresholds:** keeping the rate below the rules' limits. It fools the rules that rely on thresholds, while the model can catch the pattern from other features, such as the number of ports.

The main conclusion is that every trick that neutralizes one detector leaves traces visible to the other. This is exactly the reason we built a hybrid system. The union of the two signals raises the recall, while the handling of disagreements keeps false positives low. The non-automatic escalation of indications that come only from an anomaly (rule C2), together with the perfect precision of the signatures, limit alert fatigue.

We must, however, be honest about the limits. The model was trained on known types of attacks. Against a truly unknown (zero-day) attack, supervised learning does not guarantee detection, while unsupervised learning offers some coverage at the cost of low precision. An attacker who knows how the system works could craft traffic that statistically mimics the legitimate one, a technique called adversarial evasion (lecture 2), against which neither of the two sides is on its own robust.

Optional Task 1: Supervised versus unsupervised learning

This extension compares two fundamentally different machine-learning approaches, so as to bring out a basic security trade-off. In supervised learning (Random Forest, Logistic Regression) the model is trained with labeled examples, that is, it knows in advance which traffic is an attack and which is not. In unsupervised learning (Isolation Forest) the model sees no label, it only learns what normal traffic looks like and flags whatever deviates from it.

The difference is captured in the metrics. The two supervised models achieve clearly higher precision (0.891 for Random Forest, 0.696 for Logistic Regression), because, having seen attack examples, they draw more precise boundaries between legitimate and malicious traffic. Isolation Forest, on the contrary, shows low precision (0.397) and a high false positive rate ($FPR=0.178$), because it flags as an anomaly every unusual window, even when it is legitimate but rare behavior. This practically confirms the principle that the unusual is not identical to the malicious (lecture 3).

The finding has practical significance. In real security environments confirmed attack labels are rare and costly, which makes unsupervised learning attractive, since it does not require them. Its low precision, however, makes it unsuitable as a standalone alerting mechanism and more suitable as an auxiliary layer of monitoring and prioritization. This role it takes on in the fusion layer through rule C2, where Isolation Forest does not produce a standalone alert but only raises the severity level. A trade-off is also observed between the two supervised models, since Random Forest, being non-linear,

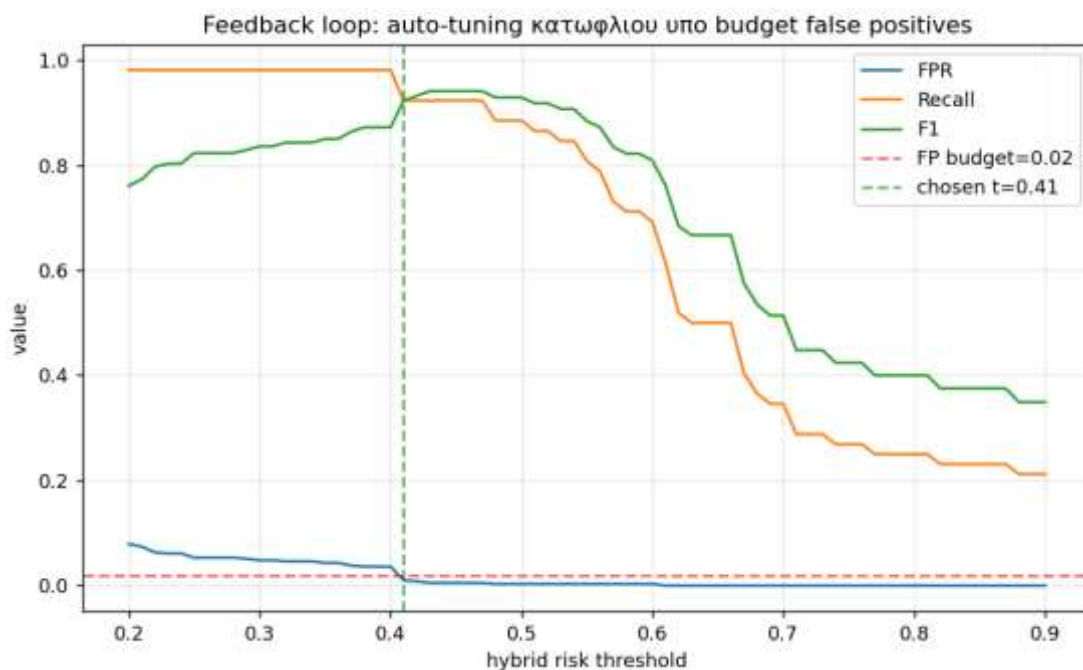
achieves better precision, while Logistic Regression is simpler and more interpretable, at the cost of more false positives.

Files implementing Optional Task 1: `ml_detector/train_model.py` trains all three models, while `ml_detector/evaluation.py` produces their comparative assessment, captured in figures 1 and 3.

Optional Task 2: Feedback loop

The threshold above which the system raises an alert is not a fixed parameter, but a choice that depends on the relative cost of a false alarm versus a missed attack (lecture 5). For this reason a feedback mechanism was implemented, which automatically adjusts the decision threshold of the hybrid risk score, so that the false positive rate stays below a predefined operational limit, here set at 2%.

The mechanism sweeps successive threshold values, measures for each the false positive rate, the recall and the F1 score, and selects the lowest value that satisfies the constraint. On the data it lands at the value 0.41, which yields a false positive rate of 0.010 with $F1=0.923$. It is a simple form of adaptive control, which could be extended so as to update the threshold based on analysts' feedback over time. The following figure captures the sweep and the operating point that was selected:



Files implementing Optional Task 2: the feedback mechanism is implemented in `ml_detector/evaluation.py` and its result is captured in the figure above.

Execution conditions and parameters

So that anyone can reproduce exactly the same results, we record the execution conditions:

- **Data:** the analysis is done offline over a single traffic file (24,110 packets), without live capture.
- **Reproducibility:** a fixed random seed (1337) in the traffic generation and the training, so that every run gives the same results.
- **Models:** Random Forest (200 trees, maximum depth 8, with class balancing), Logistic Regression and Isolation Forest, from the scikit-learn library.

The whole chain, from the traffic generation to the evaluation, runs as one through `run_all.sh` and is documented in `README.txt`. The real Snort runs separately, offline, only for the alert screenshots.

Limitations and threats to validity

In the spirit of honest reporting (lecture 7), the limits within which the conclusions hold are stated explicitly:

- **What is documented:** on the test set, Snort alone achieved precision 1.000 and recall 0.346, Random Forest $F1=0.916$, and the hybrid the best $F1=0.926$, with the labeling of the windows independent of the features.
- **Synthetic data:** the results come from controlled, synthetically generated traffic. It is not documented that the performance transfers unchanged to a production network with a different traffic distribution.
- **Range of attacks:** the metrics concern six specific types of attacks and do not generalize to unknown threats.
- **Parameter selection:** the rules' thresholds and the fusion weights were chosen empirically and would need re-tuning in another environment.
- **Window size:** we work on one-second windows. Attacks that unfold on a much smaller or much larger time scale may need a different window size.

These limitations do not negate the central finding, but delineate precisely its scope of validity and point to specific directions for future research.

Conclusion

We built a complete and repeatable framework for evaluating a hybrid intrusion detection system, which combines signature-based detection (Snort) with machine-learning-based detection. We did not stop at demonstrating detections, but treated the problem as something measurable, with a full cycle, from the design of the lab and the generation of the traffic, to the fusion of the two layers and the systematic evaluation.

As for the weakness of the individual approaches, Snort showed perfect precision but low recall (0.346), letting the slow and the stealthy attacks through. Machine learning raised the recall a lot, but introduced false positives. Most importantly, the numbers did not come out artificially perfect, precisely because we kept the truth independent of the features and the phenomenon of data leakage was avoided.

As for the solution, the hybrid system achieved the best F1 (0.926), surpassing every individual approach. The cases we examined showed in a measurable way that the two sides are complementary, not redundant. Machine learning caught 98 windows that Snort ignored, and Snort caught a stealthy attack that machine learning took for legitimate. The two optional extensions reinforced the picture, showing the role of anomaly detection as a monitoring mechanism and the way of tuning the operating point.

A steady axis of the work was the independence of the labeling from the features, the reporting of the failures and the clear delimitation of the conclusions. Beyond confirming that the hybrid combination improves detection, the work provides a rigorous and repeatable methodology for quantifying this improvement, which constitutes its main contribution.

Deliverables and execution instructions

The submission is organized in a single folder, with a clear separation between code, data and documentation. The traffic-generation code is in the `traffic_generation/` folder, the machine-learning part in `ml_detector/`, the rules in `rules/`, while the generated data are distributed across the folders `logs/`, `pcaps/` and `figures/`. Full instructions are in `README.txt`, `TESTBED_DESIGN.md`, `CASE_STUDIES.md` and `SNORT_GUIDE.md`. The table below shows which file implements each requirement:

Task	Implemented by:
1. Testbed and threat model	TESTBED_DESIGN.md, traffic_generation/build_dataset.py
2. Traffic generation and labeling	build_dataset.py, ml_detector/feature_extraction.py
3. Snort rules	rules/local.rules, rules/snort.lua, ml_detector/snort_emulator.py, rules/snort_kali.lua, rules/rules_kali.rules, logs/alert_fast.txt, screenshots/
4. ML models	feature_extraction.py, train_model.py
5. Hybrid fusion	ml_detector/hybrid_fusion.py
6. Evaluation	ml_detector/evaluation.py, CASE_STUDIES.md
7. Evasion and critical assessment	the present report (Task 7 section)
Optional task 1	train_model.py, evaluation.py
Optional task 2	evaluation.py (feedback loop)

Beyond the code, the submission includes the present report, the documentation files, the dependency list (requirements.txt) and the Snort execution guide for the screenshots.